Sistema de Acompañamiento Académico: informe de estado de implementación

Julian David Hernández y David Santiago Arteaga

Tatiana Cabrera

Algoritmos y estructura de datos

Universidad del Rosario

18 de Octubre de 2026

Este informe describe el estado actual de un sistema de gestión de solicitudes de acompañamiento académico, desarrollado en el lenguaje de C++ sin conexión a internet y sin motor de base de datos. El sistema registra usuarios, materias y solicitudes, mantiene una cola de pendientes con prioridad, permite revertir asignaciones y ordena las solicitudes con merge sort. En el cual se registra los nueve requerimientos funcionales (RF) y los cinco no funcionales (RNF) que implementa la problemática, de los RF hay cuatro cumplidos y cinco parciales o pendientes, en cuanto a los RNF de datos masivos y doble lenguaje no se realizaran por acuerdo del equipo. También se documentan los criterios de priorización que se incorporarán con el grafo de prerrequisitos y las limitaciones que conocemos.

Introducción

El programa de monitorías suelen operar con mensajes sueltos, horarios que se cruzan y decisiones de atención sin un orden, generando un favoritismo. Este proyecto propone una herramienta de escritorio que ordena ese proceso. El documento sirve de seguimiento y es útil para la sustentación: indica qué requerimientos ya funcionan, cuáles están a medias y cuáles faltan, y con qué estructuras y algoritmos se resolvió cada uno.

Descripción del problema

El enunciado identifica cuatro dificultades: las solicitudes se pierden porque llegan por canales informales; no existe un criterio explícito para decidir a quién se atiende primero, lo que genera percepción de favoritismo, los horarios se cruzan y se pierden sesiones ya asignadas, y no se detecta a tiempo al estudiante que acumula dificultades en materias encadenadas por prerrequisitos. La solución debe gestionar las solicitudes, emparejarlas con monitores según materia y disponibilidad, priorizarlas con criterios, y organizar la agenda sin cruces y permitir ver el encadenamiento de materias para anticipar el riesgo académico.

Diseño de la solución:

Organización del proyecto

El proyecto se divide en tres partes: la carpeta data, con los archivos de texto que guardan la información (usuarios.txt, materias.txt, solicitudes.txt); la carpeta include, con los archivos de cabecera (.h); y la carpeta src, con las implementaciones (.cpp). Al iniciar, main carga los tres archivos en memoria,

ordena las materias por código y reconstruye la cola con las solicitudes pendientes. Al cerrar, guarda los cambios.

Estructuras y algoritmos

Componente	Archivo	Descripción	Complejidad
Jerarquía de usuarios	Usuario.h	Usuario base con Administrador, Estudiante, Monitor (hereda de Estudiante) y Profesor	n/a
Cola con prioridad	Solicitud.h	Vector ordenado por urgencia y, a igual urgencia, por id	Inserción O(n), atención O(1)
Pila de asignaciones	Asignaciones.h	Guarda cada cambio para poder deshacerlo (LIFO)	O(1)
Bitácora	Asignaciones.h	Registro de solo adición de asignaciones y reversiones	O(1)
Búsqueda secuencial y binaria	Algoritmos.cpp	Cuentan comparaciones; la binaria exige vector ordenado	O(n) y O(log n)
Inserción, merge sort y quick sort	Algoritmos.cpp	Ordenan materias por código, nombre o créditos	$O(n^2)$, O(n log n)
Merge sort de solicitudes	Algoritmos.cpp	Urgencia, antigüedad o materia, con desempate por id	O(n log n)
Recursión de prerrequisitos	Algoritmos.cpp	Imprime la cadena de prerrequisitos	Proporcional a la cadena
Horario por materia	Materia.h	Matriz de 6 días por 28 franjas de 30 minutos (6:00 AM a 8:00 PM)	O(días × franjas)

Estado de los requerimientos funcionales

Se usan tres estados: **Cumplido** (cumple los criterios del enunciado), **Parcial** (existe una base funcional, pero faltan criterios o contiene limitaciones) y **Pendiente** (no está implementado).

RF	Requerimiento	Estado	Dónde está
RF-01	Registrar estudiantes, monitores y materias con disponibilidad	Parcial	Usuario.h, GestorArchivos, Menus.cpp
RF-02	Registrar solicitudes y conservar historial	Cumplido	Solicitud.h, menuEstudiante
RF-03	Cola de pendientes con orden de llegada y prioridad	Cumplido (en actualización)	ColaSolicitudes
RF-04	Revertir la última asignación sin alterar el historial	Cumplido	Asignaciones.cpp
RF-05	Localizar estudiante o monitor por código de forma inmediata	Pendiente	Solo existe búsqueda de materias
RF-06	Ordenar solicitudes por urgencia, antigüedad y materia	Cumplido	ordenarSolicitudes
RF-07	Jerarquía de materias por área y nivel	Pendiente	Sin implementar
RF-08	Red de prerrequisitos y materias afectadas al reprobar	Parcial	mostrarPrerrequisitos, Grafo.h (concepto)
RF-09	Agenda semanal sin cruces e informe de cobertura	Parcial	Matriz de horario de materias

RF-01. Registro de estudiantes, monitores y materias (Parcial)

Se ha implementado un archivo usuarios.txt que maneja cuatro roles y el administrador puede registrar usuarios nuevos desde el menú (opción 8), con códigos consecutivos y verificación de que el nombre de usuario no se repita. Las materias se registran con créditos, tipo, prerrequisitos y un horario semanal armado por días y bloques (opción 2). Todo se guarda en archivos de texto.

Falta por implementar la disponibilidad horaria del monitor se guarda como texto libre (por ejemplo, «Lunes a Viernes 2pm-4pm»), no como franjas que el programa pueda comparar, y el estudiante no tiene disponibilidad. Por eso el emparejamiento por horario todavía no está posible.

RF-02. Solicitudes e historial (Cumplido)

La clase Solicitud guarda estudiante, materia, urgencia, descripción y estado (Pendiente, Asignada, Atendida o Cancelada). El id consecutivo registra el orden de llegada, y solo se aceptan materias registradas. El estudiante consulta únicamente sus propias solicitudes (opción 4); las canceladas y atendidas permanecen en el historial, que se conserva entre ejecuciones y no cambia al revertir una asignación. Este cubre las HU-05 y HU-06; en esta última queda por verificar el criterio de presentar el historial en orden cronológico, que se colocó como vacío por falta de datos de prueba.

RF-03. Cola de pendientes con prioridad (Cumplido, pero con limitaciones)

ColaSolicitudes mantiene un vector ordenado por urgencia de mayor a menor y, a igual urgencia, por id menor. Ofrece encolar, frente, desencolar y eliminar. Una solicitud cancelada sale de la cola, y al iniciar el programa la cola se reconstruye con las pendientes guardadas. Comparar también por id hace que una solicitud reencolada tras deshacer una asignación recupere su lugar por antigüedad. Cubre las HU-07 y HU-08.

El único criterio es la urgencia que declara el estudiante. Se definió que los nuevos criterios, que dependen del grafo, y se describirán más adelante.

RF-04. Revertir la última asignación (Cumplido, pero con limitaciones)

Cada asignación (de materia o de solicitud) se apila antes de aplicarse. deshacerAsignacion desapila la última y restaura el estado anterior: si fue de materia, devuelve la materia previa del monitor; si fue de solicitud, la solicitud vuelve a «Pendiente», regresa a la cola y se libera la franja del monitor. La bitácora solo recibe registros nuevos, de modo que deshacer añade una línea de reversión y no borra nada. Cubre la HU-21 y la opción 7 del administrador.

Limitación conocida: si la solicitud ya fue atendida. El historial no conserva el registro de la asignación y de la reversión, además de esto, Deshacer una solicitud ya atendida descarta el registro sin dejar rastro en la bitácora.

RF-05. Búsqueda inmediata por código (Pendiente)

Solo existe búsqueda de **materias** por código, secuencial (O(n)) y binaria (O(log n)), con conteo de comparaciones (HU-26). No hay búsqueda de estudiantes ni de monitores por código, y ninguna de las dos es de tiempo constante. La solución prevista es una tabla hash con el código como llave, que también serviría para guardar la cantidad de materias dependientes de cada materia.

RF-06. Ordenamiento de solicitudes (Cumplido)

Creamos ordenarSolicitudes que usa merge sort recursivo, estable y con desempate por id, con tres criterios: urgencia (mayor primero), antigüedad (id menor primero) y materia (por código y, dentro de cada una, la más antigua). Ordena una copia, de modo que el orden de llegada original no se altera, y muestra el número de comparaciones. Está disponible para el administrador (opción 11) y el monitor (opción 5). Cubre las HU-11, HU-12 y HU-13. Para las materias también se ofrecen inserción, merge sort y quick sort (HU-25).

RF-07. Jerarquía de materias por área y nivel (Pendiente)

La clase Materia no tiene campos de área ni de nivel, y no existe una estructura de árbol. Falta definir el árbol (área, nivel, materia) y la consulta de todo lo que cuelga de un área.

RF-08. Red de prerrequisitos (Parcial)

Cada materia guarda la lista de códigos de sus prerrequisitos, y mostrarPrerrequisitos recorre la cadena hacia atrás de forma recursiva, con caso base en la materia sin prerrequisitos (HU-27). Lo que falta por implementar es el sentido contrario, dado un curso reprobado, listar las materias que dependen de él, directa o indirectamente. Eso requiere el grafo dirigido (Grafo.h solo contiene la idea). Se piensa implementar una red de prerrequisitos con un grafo dirigido **acíclico**, porque permite calcular el impacto real de perder una materia, identificando al instante todas las asignaturas futuras (directas e indirectas) que quedan bloqueadas, con esta información, el sistema de acompañamiento puede generar alertas tempranas.

RF-09. Agenda semanal e informe de cobertura (Parcial)

Se ha implementado una base para que cada materia tenga una matriz de horario, se muestra como cuadrícula de días por franjas y como lista de bloques (por ejemplo, «Jueves, 9:00 AM - 11:30 AM»), y se guarda en el archivo. No hay verificación de cruces entre horarios de monitores, no se genera una agenda semanal consolidada ni el informe de cobertura del programa.

Requerimientos no funcionales

RNF	Requerimiento	Estado	Observación
RNF-01	Conjunto de datos de al menos 500 registros	No realizado	No se acordó trabajar con datos de prueba perohay 7 materias, 4 solicitudes y 5 usuarios
RNF-02	Búsqueda por código en tiempo constante promedio	Pendiente	Hoy O(log n) en materias; se resolverá con tabla hash
RNF-03	Sin internet ni base de datos externa	Cumplido	Persistencia en archivos de texto y estructuras propias
RNF-04	Código documentado y legible por terceros	Cumplido, con observaciones	Comentarios en cabeceras y funciones, nombres descriptivos
RNF-05	Misma funcionalidad en C++ y Python	No realizado	Solo se implementó C++

RNF-03. No se usan bibliotecas de red ni motores de datos. Todo se lee y escribe con ifstream y ofstream. La ruta de los archivos se resuelve desde varias carpetas para que el programa corra desde la raíz o desde src.

RNF-04. Las cabeceras explican el propósito de cada clase y de cada función, y los puntos delicados llevan comentarios en línea (por ejemplo, por qué el pivote de quick sort es el elemento central). Queda por revisar Grafo.h, que aún es solo un comentario y describe el grafo como «cíclico», cuando debe decir acíclico.

RNF-01 y RNF-05. Por indicación del equipo docente y no se exigen los 500 registros ni la doble implementación. Esto limita la medición de eficiencia: con tan pocos datos, las diferencias entre búsqueda secuencial y binaria, o entre inserción y merge sort, solo se aprecian en el conteo de comparaciones y no en tiempo.

Criterios de priorización de solicitudes

Se definió que la urgencia de una solicitud sea una suma ponderada de criterios normalizados entre 0 y 1, con pesos que suman 1:

Prioridad = w1·P1 + w2·P2 + w3·P3

El resultado se lee como porcentaje (0 a 100 %) y las solicitudes se ordenan de mayor a menor con el algoritmo de ordenamiento ya implementado. Normalizar

permite sumar magnitudes distintas (semestre, cantidad de materias, días) sin que una domine por su escala.

P1 - Semestre del estudiante. Vale 1 si el estudiante está en los dos primeros semestres (1 y 2) o en los dos últimos (8 y 9), y 0 en otro caso. La idea es atender primero a quien apenas se adapta y a quien está cerca de graduarse.

P2 - Materias que dependen de la solicitada. Sea d la cantidad de materias que dependen de la materia de la solicitud, directa o indirectamente. Entonces P2 = d / d_max, donde d_max es el mayor valor de d del catálogo. Una materia que nadie necesita da 0 y la más cercana a la raíz da 1. Las dependientes directas pesan más que las indirectas. Como d solo cambia cuando cambian los prerrequisitos, se calcula una vez por materia y se guarda en una tabla hash (materia → d).

P3 - Orden de llegada. Se usa el tiempo de espera y no el id crudo: P3 = min(espera, T) / T, con T un tope de 7 días. Mientras más espera una solicitud, más sube, y ninguna queda sin atender. Si dos puntajes coinciden, va primero el id menor.

Criterios opcionales. Riesgo académico (el estudiante reprobó o repite la materia) y cercanía de evaluación (fecha de parcial declarada). Ambos necesitan datos nuevos. Con cuatro o cinco criterios en total el modelo sigue siendo explicable en la sustentación; más lo harían difícil de defender.

Cambios que implica en el código. Solicitud necesita una fecha de creación para calcular la espera; el semestre ya está disponible en Estudiante; Grafo debe calcular d por materia; y ColaSolicitudes::encolar pasará de comparar la urgencia declarada a comparar el puntaje. Además, la urgencia que hoy elige el estudiante (se dejó temporalmente para pruebas) debe dejar de ser un dato de entrada, porque es justo lo que alimenta la percepción de favoritismo.

Estado de las historias de usuario

Se definieron alrededor de 30 historias de usuario (HU): 15 cumplidas, 9 parciales y 6 faltantes. Las parciales tienen limitaciones o criterios de aceptación sin cubrir; las faltantes aún no se implementan.

HU	Descripción	RF asociado
HU-05	Registrar solicitud de acompañamiento	RF-02
HU-06	Consultar historial del estudiante	RF-02
HU-07	Mantener cola de solicitudes pendientes	RF-03

HU-08 Priorizar solicitudes con criterios	RF-03
HU-11 Ordenar solicitudes por urgencia	RF-06
HU-12 Ordenar solicitudes por antigüedad	RF-06
HU-13 Ordenar solicitudes por materia	RF-06
HU-20 Iniciar sesión según rol	Sin RF específico
HU-21 Asignar materia a un monitor	RF-04
HU-22 Cancelar solicitud pendiente	RF-02
HU-25 Ordenar materias con distintos algoritmos	RF-06
HU-26 Buscar materia por código (secuencial y binaria)	RF-05
HU-27 Ver la cadena de prerrequisitos	RF-08
HU-28 Consultar información como profesor	Sin RF específico
HU-29 Terminar la solicitud asignada	Sin RF especifico

Observaciones y limitaciones

- La HU-8 En un futuro será nuevamente modificada debido al uso de grafos para ser útil en la red de requisitos
- La HU-30 La bitácora no se guarda en archivo
- La HU-23 No se avisa si el programa no cuenta con carpetas u archivos necesarios antes de iniciarlo completamente
- La HU-01 Falta la disponibilidad horaria del estudiante.
- La HU-26 cuenta como cumplida en su alcance (materias, secuencial y binaria), pero no satisface el RF-05 ni el RNF-02.
- Las asignaciones activas y la pila de deshacer no se persisten; solo se conserva el estado de las solicitudes.
- El archivo usuarios.txt guarda las contraseñas en texto plano. Es aceptable para un prototipo académico, pero no para un uso real.
- Algunas materias de prueba no tienen horario (por ejemplo, CAL106 y HUM301), lo que debe tenerse en cuenta al construir la agenda.

Trabajo pendiente

En orden aproximado de dependencia: (1) implementar el grafo de prerrequisitos y el cálculo de d por materia, que habilita el RF-08 y el criterio P2; (2) añadir la fecha a las solicitudes y reemplazar la urgencia declarada por el puntaje ponderado en la cola; (3) crear la tabla hash por código para estudiantes y monitores (RF-05, RNF-02); (4) estructurar la disponibilidad de monitores en franjas y construir la agenda sin cruces con el informe de cobertura (RF-01 y RF-09); (5) definir el árbol de áreas y niveles (RF-07).

Conclusiones

El sistema ya cubre el flujo central: un estudiante crea una solicitud, queda en una cola con orden definido, el administrador la asigna a un monitor, la asignación se puede revertir y todo queda en la bitácora y en archivos. Esa parte cumple cuatro de los nueve RF. Lo pendiente es la que más aporta al problema original: el criterio de prioridad basado en el grafo, la búsqueda inmediata por código y la agenda sin cruces. El diseño actual permite añadirlos sin reescribir lo existente, pues el ordenamiento, la cola y la persistencia ya están separados por archivo.

Trello:

https://trello.com/invite/b/6ac315d84e1ba0ef80bc7bef/ATTIedb1e7aab1c1a42131e5428c5ddfa0b4BA44F5CC/algoritmos-proyecto